

Valmet Hackathon – Field Device Detector

Collaborative plan for two contributors to build a high-impact demo that detects industrial field devices in 2D/3D, aligns them to the plant's 3D Gaussian splat, and ships a polished demo and pitch.

Owners

- A (2D/Vision Lead):
- B (3D/SLAM + UI Lead):

Goals

- Accurate, fast detection of field devices in videos and photos.
- 3D localization of devices aligned to the provided PLY scene(s).
- Usable demo app with synchronized video overlay and 3D viewer.
- Clear business story: "Ask Your Plant" assistant and inventory outputs.

Success Metrics (Hackathon-target)

- $mAP@.5 \geq 0.6$ on held-out frames; recall ≥ 0.7 for key classes.
- Real-time or near-real-time fast path (≥ 10 FPS on CPU).
- 3D alignment RMSE < 5 cm on landmarks; triangulation reprojection error < 2 px median.
- Demo UX: < 30 sec to answer "find all hand valves near tank".

Assets (local repo)

- PLY: `PLY files/pilot_plant_devices.ply`, `PLY files/lounge_area.ply`, `PLY files/library.ply`, `PLY files/cellar_devices.ply`
- Videos: `Videos/Pilot_plant.mp4`, `Videos/Coffee_place.mp4`, `Videos/corridor.mp4`
- Baseline TFJS detector: `Tensorflow AutoML model/model-..._model.json`, shards, and `..._dict.txt` with labels: background, positioner, handvalve, motor, measurement
- Brief: `Valmet challenge.pdf`

High-Level Architecture

- Fast Path (on-device): TFJS AutoML detector for four known classes.
- Open-Vocab Path: GroundingDINO or OWL-ViT + SAM 2 for promptable zero-shot detection and masks.
- Tracking: ByteTrack or BoT-SORT for stable IDs over time.
- Camera Poses: COLMAP or ORB-SLAM2 from video to get intrinsics/extrinsics.

- 3D Triangulation: Reproject and triangulate tracked detections; align to PLY via Open3D RANSAC+ICP.
- Assistant: Optional VLM integration (featherless.ai) for visual QA and natural-language queries.
- Demo App: Three.js PLY viewer + video pane + overlays + “Ask Your Plant”.

Workstreams and Ownership

- Workstream A (Owner A)
 - 2D detection (TFJS baseline, open-vocab integration), tracking, thresholds, data augmentation.
 - JSON export: per-frame detections, tracked IDs, confidences, masks (if available).
- Workstream B (Owner B)
 - SfM/SLAM, triangulation, PLY alignment, 3D pin rendering, Three.js viewer, UX polish.
 - “Ask Your Plant” UI, filters, counts, device list; optional VLM integration.

Folder Structure (proposed)

- `web/` – Demo app (Three.js viewer, TFJS inference, controls)
- `scripts/` – Data prep (keyframe extraction, COLMAP helpers), evaluation scripts
- `experiments/` – Notebooks/configs for model tests and threshold tuning
- `outputs/` – Generated artifacts (detections JSON, camera poses, 3D pins)

Day-by-Day Plan (4 days)

Day 1 (Baseline working demo)

- A: Wire TFJS model in a basic web page; run inference on `Videos/Pilot_plant.mp4`; draw boxes; export per-frame JSON.
- B: Stand up Three.js viewer loading a PLY; add controls; design overlay schema for 3D pins.
Deliverable: Minimal demo showing 2D boxes and a 3D scene side-by-side.

Day 2 (Open-vocab + Tracking)

- A: Add ByteTrack/BoT-SORT; integrate GroundingDINO or OWL-ViT; add SAM 2 masks; class prompt list.
- B: Define JSON schemas for tracks and masks; implement a video timeline with overlay; class filters.
Deliverable: Stronger detections, stable track IDs, toggle between detectors.

Day 3 (3D Localization)

- B: Run COLMAP/ORB-SLAM2 to recover camera poses; perform alignment to PLY (Open3D ICP); compute 3D pins.
- A: Provide centroid points per track; add reprojection error checks; assist with outlier filtering (RANSAC).
Deliverable: 3D pins placed in the scene; scrub video to see synchronized 2D/3D.

Day 4 (Assistant + Polish + Pitch)

- A: Calibrate thresholds; produce metrics; finalize JSON exports.
 - B: "Ask Your Plant" search; insights view (counts, heatmap); polish UX; script and record demo video; finalize slides.
- Deliverable: Polished demo video and pitch deck with metrics.

Detailed Checklists

Workstream A (2D/Vision)

- ☐ Load TFJS AutoML model from `Tensorflow AutoML model/`
- ☐ Fast inference loop on sampled frames (configurable stride)
- ☐ Draw boxes + confidence; per-class threshold sliders
- ☐ Export `outputs/detections/{video_name}.json` with boxes, scores, class, frame idx
- ☐ Integrate tracker (ByteTrack/BoT-SORT) → track IDs + track scores
- ☐ Open-vocab detector (GroundingDINO or OWL-ViT) with curated prompts
- ☐ SAM 2 masks for precise centroids (optional if time)
- ☐ Temporal smoothing + NMS; outlier suppression via geometry priors
- ☐ Metrics: mAP@.5, recall, precision on hold-out frames; per-class calibration
- ☐ Provide per-track centroid sequence for triangulation

Workstream B (3D/SLAM + UI)

- ☐ Three.js viewer loading `PLY files/*.ply` with controls (orbit, zoom, section)
- ☐ Video pane with overlay and scrubber; sync with detections JSON
- ☐ Schema: `outputs/tracks/{video}.json` (poses, intrinsics),
`outputs/3d_pins/{scene}.json`
- ☐ COLMAP/ORB-SLAM2: recover camera intrinsics/extrinsics from video
- ☐ Align SfM sparse cloud to PLY via Open3D RANSAC+ICP; log RMSE
- ☐ Triangulate device centroids per track; RANSAC to filter outliers; compute covariances
- ☐ Render 3D pins with uncertainty cones; link pin ↔ 2D track ID
- ☐ Insights: counts per class, density heatmap, top uncertainties
- ☐ "Ask Your Plant" filters and NL query box; optional VLM integration
- ☐ Export inventory CSV/JSON (class, 3D coord, conf, track meta)

Schemas (initial)

- Detections per frame (`outputs/detections/pilot_plant.json`)

```
{
  "video": "Videos/Pilot_plant.mp4",
  "frames": [
    { "i": 0, "dets": [
      { "id": 12, "cls": "handvalve", "conf": 0.78, "bbox": [x, y, w,
```

```

    ] }
  ]
}

```

- Camera + tracks(`outputs/tracks/pilot_plant_tracks.json`)

```

{
  "intrinsics": {"fx": ..., "fy": ..., "cx": ..., "cy": ...},
  "poses": [{"i": 0, "Tcw": [16] }],
  "tracks": [{"id": 12, "frames": [{"i": 0, "u": x, "v": y}]}]
}

```

- 3D pins(`outputs/3d_pins/pilot_plant_devices.json`)

```

[{"id": 12, "cls": "handvalve", "p_world": [X, Y, Z], "conf": 0.73, '

```

Evaluation & Reporting

- Offline evaluation script to compute mAP/recall per class.
- SLAM/ICP logs: reprojection error distributions, ICP RMSE before/after.
- Demo dashboard: live FPS, counts, uncertainty list.

Risks & Mitigations

- Limited training data → heavy augmentation; open-vocab detectors; VLM re-ranking.
- SLAM instability → chunk video, more keyframes, manual 3-point alignment fallback in UI.
- Performance → fast path default; precompute heavy steps; caching.
- Domain shift → prompt engineering, synonym expansion, per-scene calibration.

Deliverables

- Demo video (2–3 min): problem, approach, live demo, impact.
- Slides: challenge, architecture, metrics, demo highlights, business fit.
- Repo: web app, scripts, configs, outputs samples, short README.

Collaboration Guidelines

- Use checkboxes above and add initials/date when completing (e.g., "[x] ... – A 2025-10-14").
- Keep schemas stable; if changing, version the filename (e.g., `...v2.json`) and note in "Decision Log".
- Prefer adding links to small output samples in `outputs/` for reviewers.

Decision Log

- 2025-10-14: Chose TFJS baseline + GroundingDINO for open-vocab. Owner: A.

- 2025-10-14: COLMAP primary; ORB-SLAM2 fallback; Open3D ICP. Owner: B.

Open Questions

- Target hardware/GPU availability for COLMAP and open-vocab models?
- Preferred scene among provided PLYs for demo focus?
- Any device taxonomy or spec sheets to seed prompts and RAG?

Pitch Outline (for slides)

- Problem & impact → “Humans spot devices in 1s; now AI does too.”
- Data & constraints → videos, PLY splats, few photos.
- Architecture → dual-path detector, tracking, 3D alignment, assistant.
- Demo → 2D overlay + 3D pins; counts; Ask Your Plant.
- Metrics → accuracy, speed, alignment.
- Business fit → remote support/training; Industrial Metaverse ready.